2. Manejo de Transacciones

a. Definición y Objetivos

Una transacción es un conjunto de operaciones SQL que se ejecutan como una única unidad lógica e indivisible de trabajo. O cumple todo o no cumple nada. Está delimitada por:

BEGIN TRANSACTION --- inicio.

COMMIT TRANSACTION --- confirma todo si salió bien.

ROLLBACK TRANSACTION --- deshace todo si algo fallo.

Toda transacción debe cumplir las propiedades ACID:

*Atomicidad: Todo se ejecuta o nada se ejecuta. No puede quedar a medias.

*Consistencia: La Base de Datos pasa de un estado valido a otro estado valido.

*Aislamiento: Una transacción no afecta a otra hasta que termine.

*Durabilidad: Una vez hecho COMMIT, los datos quedan guardados, aunque se caiga el sistema.

¿Por qué utilizarlas? ¿Cuál es su objetivo?

El objetivo es garantizar la integridad y consistencia de los datos cuando hay varias operaciones que dependen entre si.

En nuestro sistema, sistema-venta-libros, un pedido no es una sola tabla. Al crear un pedido tenemos que hacer 4 cosas a la vez.

1- Insertar en PEDIDO
2- Insertar en PEDIDO_DETALLE
3- Insertar en PAGO
4- Actualizar el stock en PUBLICACION

Sin transacción, si falla el paso 3(por ejemplo, falla el pago), nos quedaría el pedido y el detalle creados, pero sin pago y con stock descontado. La base queda inconsistente y corrupta.

Con transacción, si cualquiera de los 4 pasos falla, se ejecuta ROLLBACK y vuelve todo al estado inicial, como si nunca hubiera empezado. Si todo sale bien, se hace COMMIT y se guardan los 4 cambios juntos.

En resumen: Usamos transacciones para evitar datos a medias y para asegurar que una operación de negocio critica sea todo o nada.

Descripción del procedimiento `sp_crear_pedido` y aplicación de transacciones

El procedimiento `dbo.sp_crear_pedido` automatiza la creación completa de un pedido de venta de libros, garantizando que las 4 operaciones críticas se realicen de forma atómica.

Flujo del procedimiento:

```
USE [sistema-venta-libros];
GO

CREATE OR ALTER PROCEDURE sp_crear_pedido
    @p_usuario_id INT,
    @p_ciudad_id INT,
    @p_envio_calle NVARCHAR(255),
    @p_envio_altura NVARCHAR(20),
    @p_envio_codigo_postal NVARCHAR(20),
    @p_publicacion_id INT,
    @p_cantidad INT,
    @p_pago_metodo_id INT,
    @p_envio_piso NVARCHAR(20) = NULL,
    @p_envio_departamento NVARCHAR(20) = NULL,
    @p_envio_referencia NVARCHAR(500) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @v_precio DECIMAL(10,2);
    DECLARE @v_stock INT;
    DECLARE @v_disponible BIT;
    DECLARE @v_pedido_id INT;
    DECLARE @v_importe DECIMAL(10,2);
    DECLARE @v_pedido_estado_id INT;
    DECLARE @v_pago_estado_id INT;

    BEGIN TRY
        BEGIN TRANSACTION; -- 1. Inicio de la unidad atómica

            -- 2. VALIDACIONES (si falla alguna, salta al CATCH)

        IF @p_cantidad IS NULL OR @p_cantidad <= 0
        BEGIN
            RAISERROR('ERROR: La cantidad debe ser mayor a 0', 16, 1);
        END

        IF NOT EXISTS (SELECT 1 FROM USUARIO WHERE usuario_id = @p_usuario_id AND
fecha_eliminacion IS NULL)
        BEGIN
```

```
            RAISERROR('ERROR: Usuario inexistente o eliminado', 16, 1);
        END

        IF NOT EXISTS (SELECT 1 FROM CIUDAD WHERE ciudad_id = @p_ciudad_id)
        BEGIN
            RAISERROR('ERROR: Ciudad inexistente', 16, 1);
        END

        SELECT @v_precio = precio, @v_stock = stock, @v_disponible = disponible
        FROM PUBLICACION WITH (UPDLOCK, ROWLOCK)
        WHERE publicacion_id = @p_publicacion_id AND fecha_eliminacion IS NULL;

        IF @v_precio IS NULL
        BEGIN
            RAISERROR('ERROR: Publicacion inexistente o eliminada', 16, 1);
        END

        IF @v_disponible = 0
        BEGIN
            RAISERROR('ERROR: Publicacion no disponible', 16, 1);
        END

        IF @v_stock < @p_cantidad
        BEGIN
            RAISERROR('ERROR: Stock insuficiente', 16, 1);
        END

        SELECT TOP 1 @v_pedido_estado_id = pedido_estado_id FROM PEDIDO_ESTADO WHERE
nombre = 'Pendiente';
        IF @v_pedido_estado_id IS NULL SET @v_pedido_estado_id = 1;

        SELECT TOP 1 @v_pago_estado_id = pago_estado_id FROM PAGO_ESTADO WHERE nombre
= 'Pendiente';
        IF @v_pago_estado_id IS NULL SET @v_pago_estado_id = 1;
```

-- 3. OPERACIONES DE NEGOCIO

```
        INSERT INTO PEDIDO (fecha_pedido, envio_calle, envio_altura, envio_piso,
envio_departamento, envio_codigo_postal, envio_referencia, usuario_id,
pedido_estado_id, ciudad_id)
        VALUES (GETDATE(), @p_envio_calle, @p_envio_altura, @p_envio_piso,
@p_envio_departamento, @p_envio_codigo_postal, @p_envio_referencia, @p_usuario_id,
@v_pedido_estado_id, @p_ciudad_id);

        SET @v_pedido_id = SCOPE_IDENTITY();
        SET @v_importe = @v_precio * @p_cantidad;

        INSERT INTO PEDIDO_DETALLE (cantidad, precio_unitario, pedido_id,
publicacion_id)
        VALUES (@p_cantidad, @v_precio, @v_pedido_id, @p_publicacion_id);

        UPDATE PUBLICACION SET stock = stock - @p_cantidad WHERE publicacion_id =
@p_publicacion_id;

        INSERT INTO PAGO (importe, fecha_creacion, pago_estado_id, pago_metodo_id,
pedido_id)
        VALUES (@v_importe, GETDATE(), @v_pago_estado_id, @p_pago_metodo_id,
@v_pedido_id);
```

-- COMMIT TRANSACTION; -- 4. Todo salió bien, se confirma

```
        COMMIT TRANSACTION;

        SELECT CONCAT('PEDIDO CREADO OK - ID: ', @v_pedido_id, ' IMPORTE: ',
@v_importe) AS Resultado;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
```

ROLLBACK TRANSACTION;
-- 5. Algo falló, se deshace todo

```
        -- Esto devuelve el error real
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END
GO
```

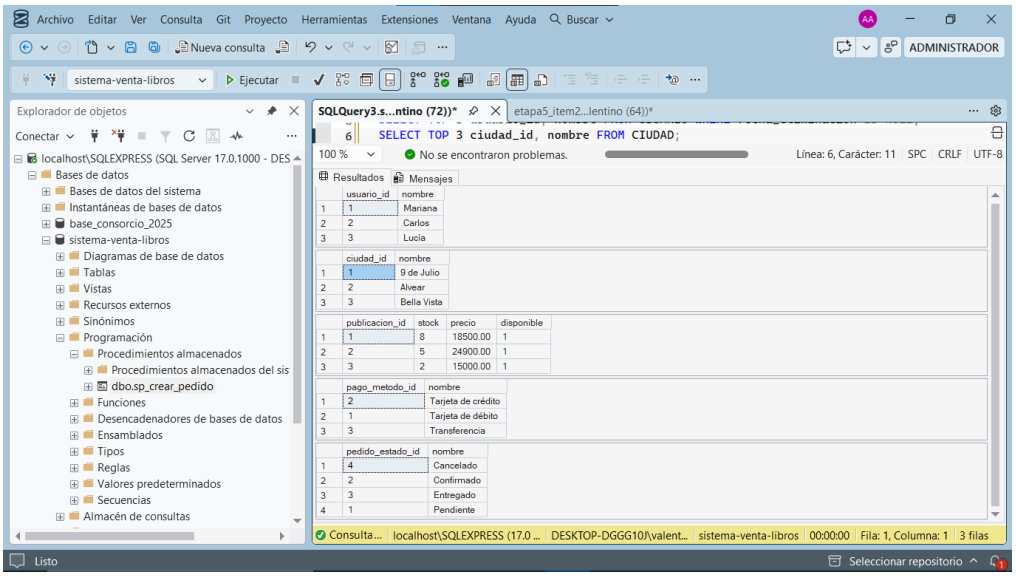
¿Cómo aplica cada instrucción?

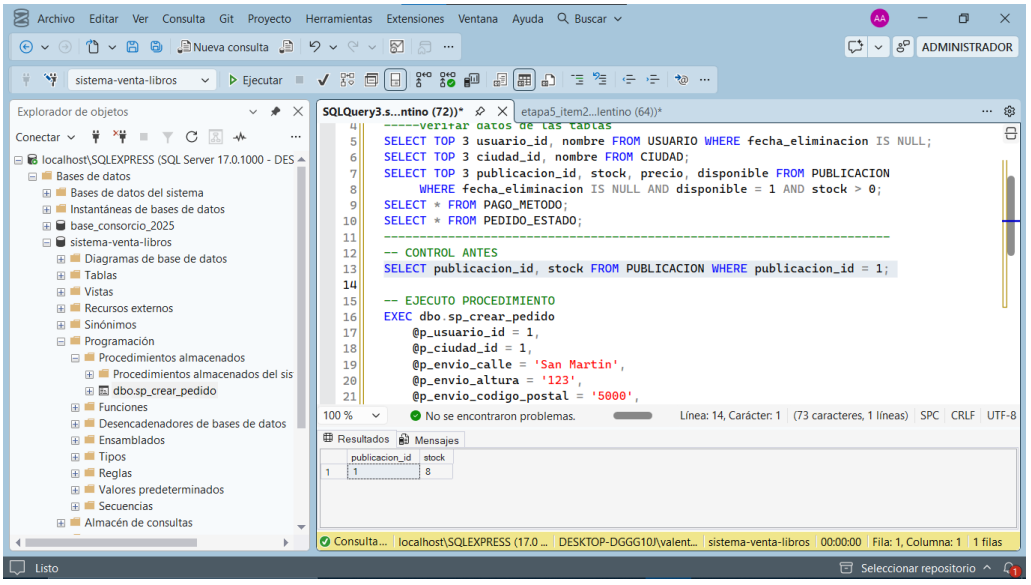
- *BEGIN TRANSACTION:* Marca el inicio. A partir de acá, SQL Server guarda todos los cambios en forma temporal hasta decidir si los confirma o no.

- CASO VALIDO

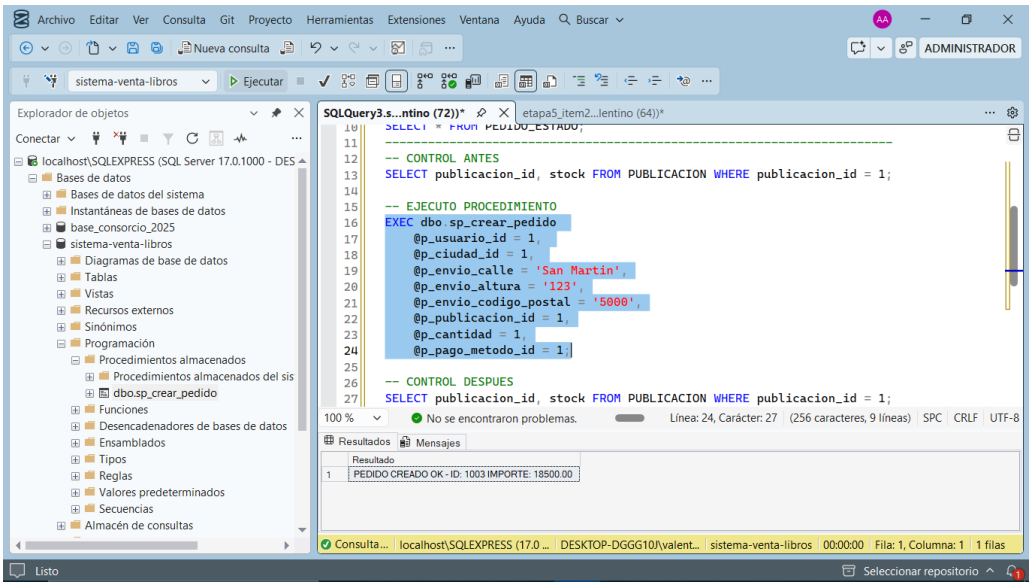
```
-----verifar datos de las tablas antes de la ejecucion de la venta

SELECT TOP 3 usuario_id, nombre FROM USUARIO WHERE fecha_eliminacion IS NULL;
SELECT TOP 3 ciudad_id, nombre FROM CIUDAD;
SELECT TOP 3 publicacion_id, stock, precio, disponible FROM PUBLICACION
     WHERE fecha_eliminacion IS NULL AND disponible = 1 AND stock > 0;
SELECT * FROM PAGO_METODO;
SELECT * FROM PEDIDO_ESTADO;
```

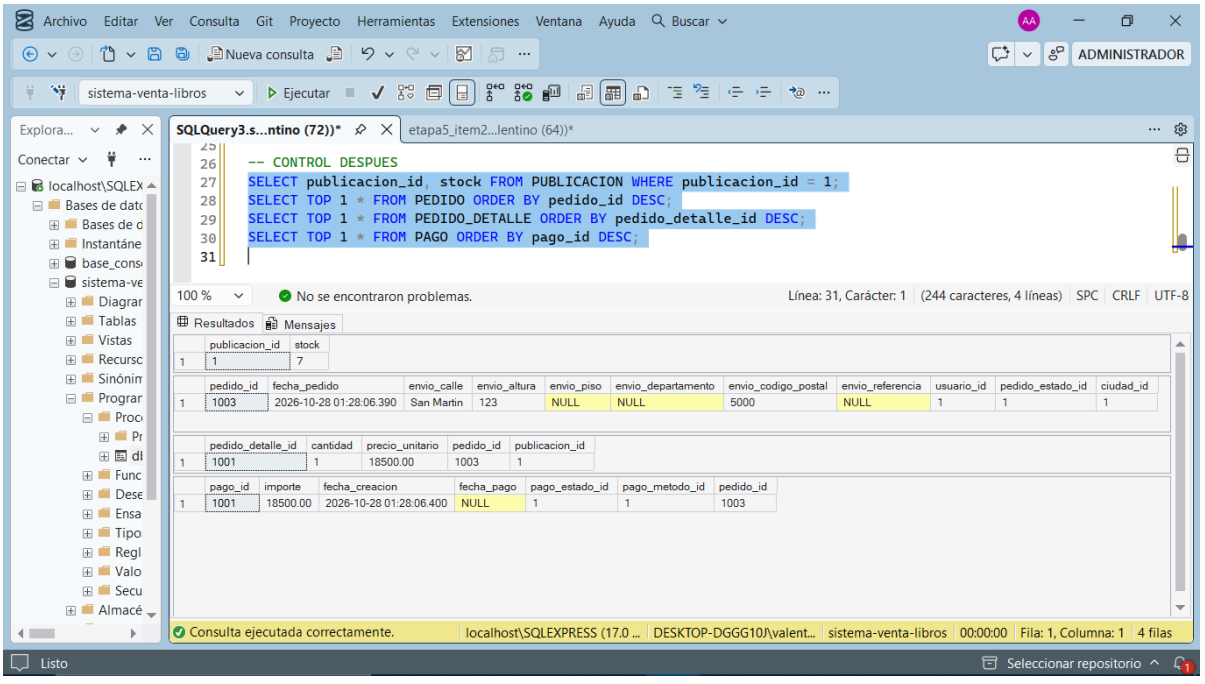





```sql
-- EJECUTO PROCEDIMIENTO
EXEC dbo.sp_crear_pedido
    @p_usuario_id = 1,
    @p_ciudad_id = 1,
    @p_envio_calle = 'San Martin',
    @p_envio_altura = '123',
    @p_envio_codigo_postal = '5000',
    @p_publicacion_id = 1,
    @p_cantidad = 1,
    @p_pago_metodo_id = 1;
```

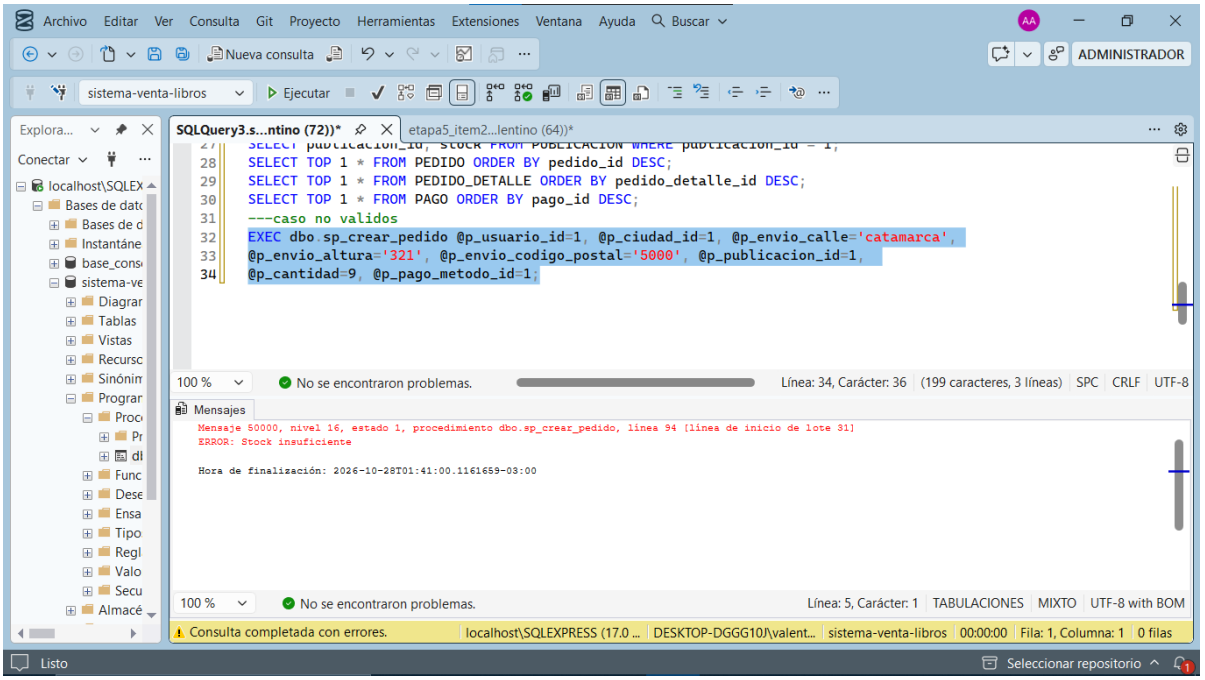


COMMIT TRANSACTION: Se ejecuta solo si las validaciones y los 4 INSERT/UPDATE fueron exitosos. En nuestro caso:
Caso válido probado: `Stock 8 -> venta 1 -> Stock 7`. Se crearon registros en `PEDIDO`, `PEDIDO_DETALLE` y `PAGO`. Todo quedó persistido.



CASO NO VALIDO

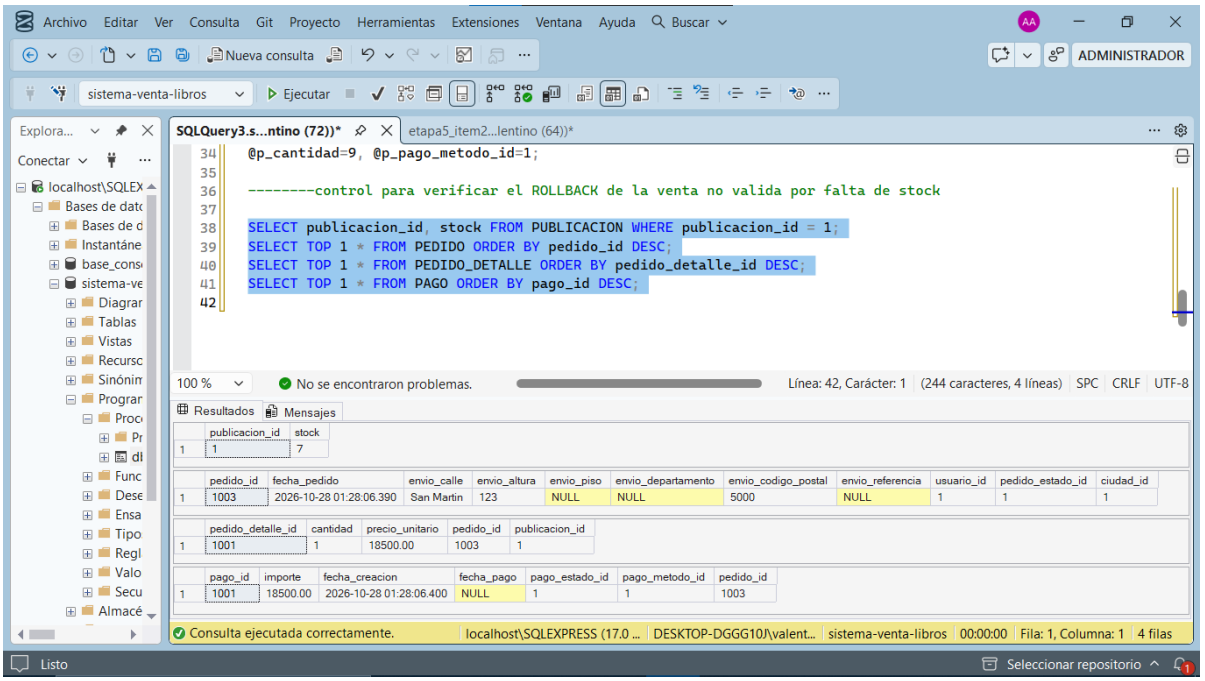
```
USE [sistema-venta-libros];
GO
---caso no validos----------------------
EXEC dbo.sp_crear_pedido @p_usuario_id=1, @p_ciudad_id=1, @p_envio_calle='catamarca',
@p_envio_altura='321', @p_envio_codigo_postal='5000', @p_publicacion_id=1,
@p_cantidad=9, @p_pago_metodo_id=1;
```



- *ROLLBACK TRANSACTION:* Se ejecuta dentro del `CATCH` si ocurre cualquier error (ej: `RAISERROR('Stock insuficiente')`). Su función es deshacer todo lo que se hizo dentro de la transacción.

```
--------control para verificar el ROLLBACK de la venta no valida por falta de stock

SELECT publicacion_id, stock FROM PUBLICACION WHERE publicacion_id = 1;
SELECT TOP 1 * FROM PEDIDO ORDER BY pedido_id DESC;
SELECT TOP 1 * FROM PEDIDO_DETALLE ORDER BY pedido_detalle_id DESC;
SELECT TOP 1 * FROM PAGO ORDER BY pago_id DESC;
```



Caso no válido probado: Intento de venta de 9 unidades con stock 7. El sistema detectó stock insuficiente, disparó el error y ejecutó ROLLBACK. Resultado: el stock se mantuvo en 7 y no se insertó ningún pedido nuevo, manteniendo la integridad.

- *TRY...CATCH + @@TRANCOUNT:* Es el mecanismo de control. Si hay error, `@@TRANCOUNT > 0` nos dice que hay una transacción abierta y hay que cerrarla con ROLLBACK.

Con esto demostramos Atomicidad: en el caso no válido no quedó ni el pedido sin pago, ni el stock descontado sin pedido.